

# Software Engineering

## Lecture 1: Introduction to Software Engineering

---

Tomas Raila

Institute of Computer Science, Vilnius University

2026-09-08

# Topics for today

What is software engineering?

What are we actually building?

A brief history

The software crisis

Why building software is different from traditional engineering

Optimizing for learning and managing complexity

Summary

# What is software engineering?

---

## IEEE / SWEBOK

The application of a systematic, disciplined, quantifiable approach to the development, operation, and maintenance of software.

# Many definitions

## **IEEE / SWEBOK**

The application of a systematic, disciplined, quantifiable approach to the development, operation, and maintenance of software.

## **David Parnas**

Multi-person development of multi-version programs.

# Many definitions

## IEEE / SWEBOK

The application of a systematic, disciplined, quantifiable approach to the development, operation, and maintenance of software.

## David Parnas

Multi-person development of multi-version programs.

## Titus Winters et al., *Software Engineering at Google* (2020)

Software engineering is programming integrated over time.

# Many definitions

## **IEEE / SWEBOK**

The application of a systematic, disciplined, quantifiable approach to the development, operation, and maintenance of software.

## **David Parnas**

Multi-person development of multi-version programs.

## **Titus Winters et al., *Software Engineering at Google* (2020)**

Software engineering is programming integrated over time.

## **David Farley, *Modern Software Engineering* (2021)**

The application of an empirical, scientific approach to finding efficient, economic solutions to practical problems in software.

## What they have in common

- **Systematic** — structured, planned and repeatable process, not improvisation
- **Over time** — the system must survive change
- **With other people** — more than one head, more than one version
- **Under constraints** — budget, deadline, staff, risk
- **For someone else** — you are solving *their* problem

Programming makes a computer do something.  
Engineering makes it keep doing it, at scale, as things change.



**What are we actually building?**

---

You have all written programs.  
Why do we need a whole discipline for that?

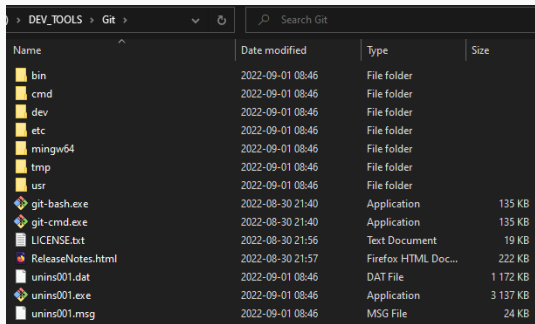
# A program

```
class HelloWorld {  
    public static void main(String[] args) {  
        System.out.println("Hello, World!");  
    }  
}
```

- Written by one person
- Runs on your machine
- Used by the author
- Lives for hours or days
- “Done” when it runs

This is probably what the previous courses so far have asked of you.

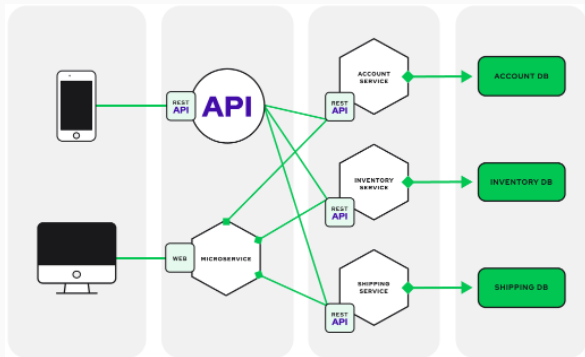
# A software product



| Name              | Date modified    | Type                | Size     |
|-------------------|------------------|---------------------|----------|
| bin               | 2022-09-01 08:46 | File folder         |          |
| cmd               | 2022-09-01 08:46 | File folder         |          |
| dev               | 2022-09-01 08:46 | File folder         |          |
| etc               | 2022-09-01 08:46 | File folder         |          |
| mingw64           | 2022-09-01 08:46 | File folder         |          |
| tmp               | 2022-09-01 08:46 | File folder         |          |
| usr               | 2022-09-01 08:46 | File folder         |          |
| git-bash.exe      | 2022-08-30 21:40 | Application         | 135 KB   |
| git-cmd.exe       | 2022-08-30 21:40 | Application         | 135 KB   |
| LICENSE.txt       | 2022-08-30 21:56 | Text Document       | 19 KB    |
| ReleaseNotes.html | 2022-08-30 21:57 | Firefox HTML Doc... | 222 KB   |
| unins001.dat      | 2022-09-01 08:46 | DAT File            | 1 172 KB |
| unins001.exe      | 2022-09-01 08:46 | Application         | 3 137 KB |
| unins001.msg      | 2022-09-01 08:46 | MSG File            | 24 KB    |

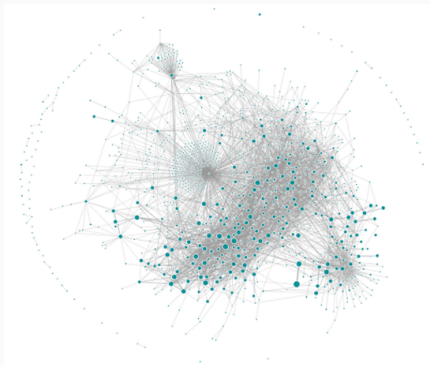
- Installed on machines you will never see
- Used by people you will never meet
- Requirements can be uncertain or changing
- Documented, supported, versioned
- Has to be maintained for months or years

# A software system

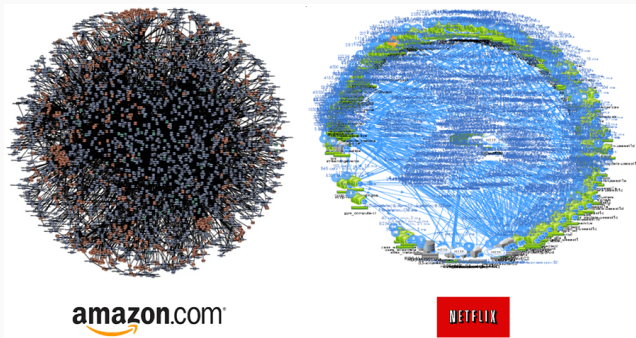


- Many components, many teams
- Interfaces and contracts between them
- Nobody understands all of it
- Parts change independently
- Failures can be expensive and/or dangerous
- Never “finished”

# A software system

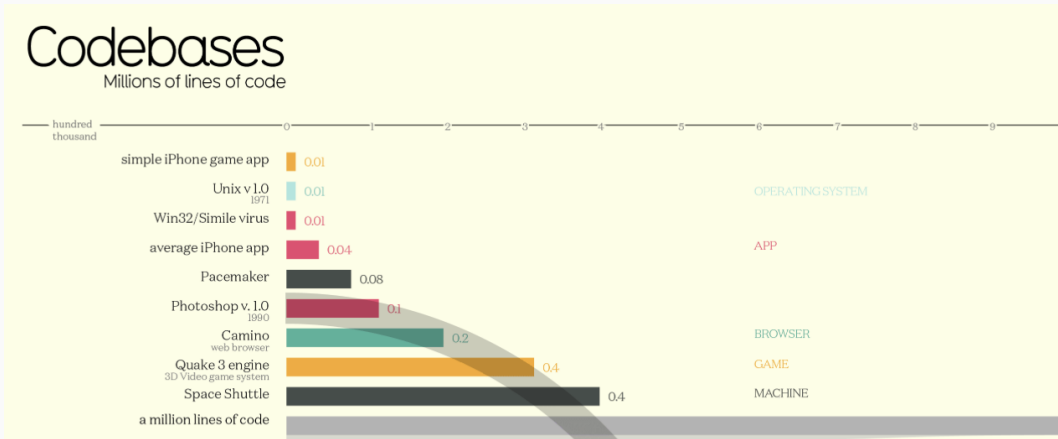


**Figure 1:** Uber's microservice architecture  
([source](#))



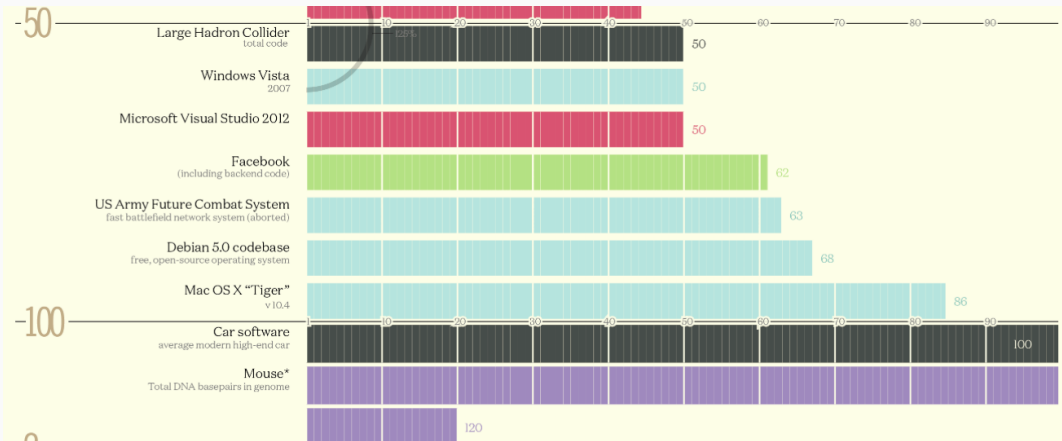
**Figure 2:** Amazon's and Netflix's microservice architectures  
([source](#))

# Codebase sizes



**Figure 3:** Codebase sizes of some well-known software systems (source)

# Codebase sizes

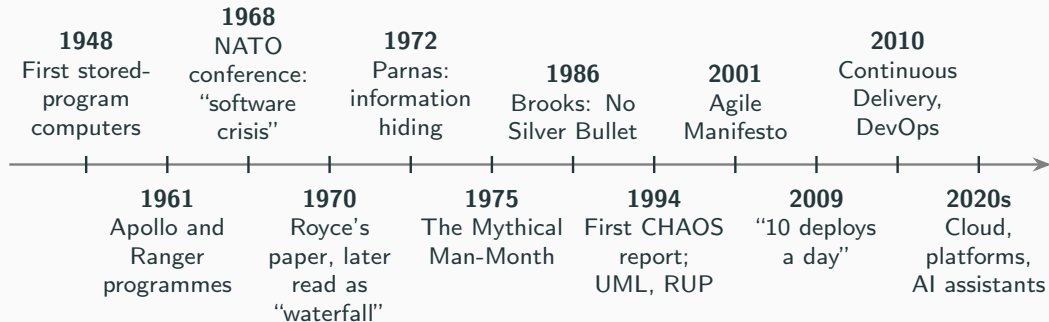




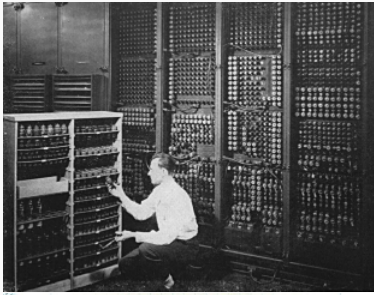
## A brief history

---

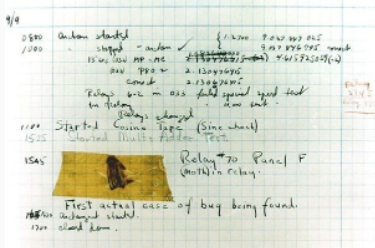
# Milestones



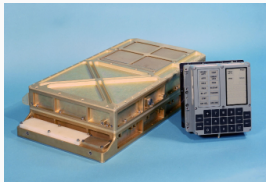
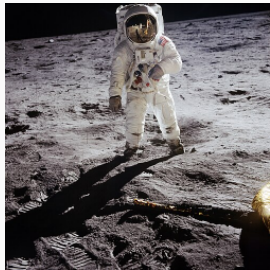
## 1940s - 1950s



- First digital computers
- Hardware was expensive; programmers were cheap
- Programming was treated as a craft, learned by apprenticeship
- No shared vocabulary, no methods, no separate profession
- Software bugs = actual insects :)



# Apollo program: the first really hard software project



- Margaret Hamilton led the development of the on-board flight software
- Coined the term “**software engineering**”
- Apollo guidance computer: about 72 KB of read-only memory, 4 KB of RAM
- Roughly 145,000 lines across the Command and Lunar Module programs
- No second chances, no patches after launch
- Lives depended on it

# NATO conference, Garmisch, Germany, 1968



*Software engineering is the establishment and use of sound engineering principles in order to obtain economically, software that is reliable, maintainable and works efficiently on real machines.*

– Software Engineering: A Report on a Conference Sponsored by the NATO Science Committee, NATO, 1969

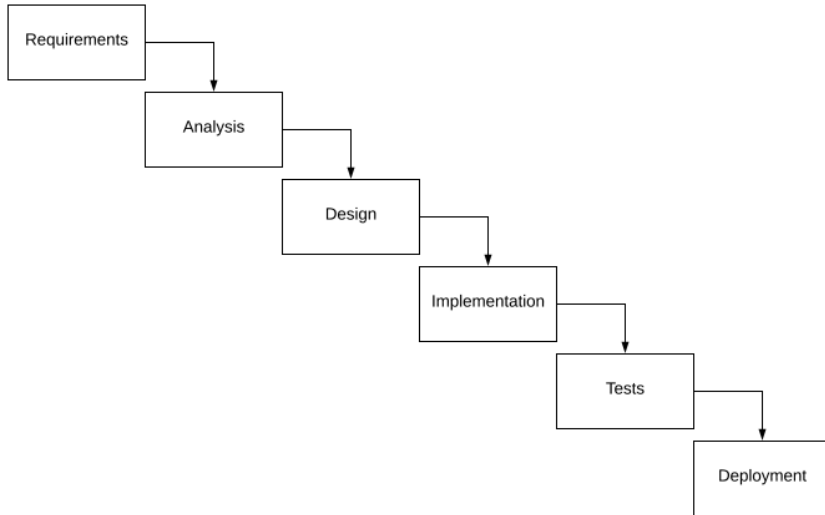
- Around 50 international experts, including prominent computer scientists
- Acknowledged the "software crisis" of unreliable and over-budget software projects
- This conference is usually taken as the origin of the discipline.

## 1970s–80s: first attempts at structure

- **Structured programming** — write code from simple, predictable building blocks (if/else, loops) instead of jumping around unpredictably
- **Modularity** (Parnas, 1972) — split a system into independent pieces, each hiding its own details
- **Object-oriented programming** (1980s) — model a program as a set of objects that send each other messages
- **Waterfall** — plan the whole system up front, then build it step by step

The first serious attempts to make building software more systematic.

# Waterfall model



# Waterfall model: a different perspective



How the customer explained it



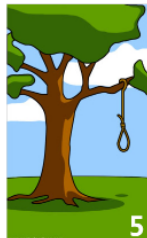
How the project leader understood it



How the analyst designed it



How the programmer wrote it



What the beta testers received



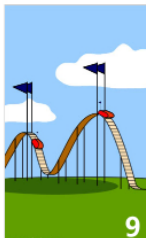
How the business consultant described it



How the project was documented



What operations installed



How the customer was billed



How it was supported



What marketing advertised



What the customer really needed



## 1990s–2000s: Agile development



- By the 1990s, “being systematic” had turned into thick manuals and heavy documentation
- Projects were still slow, and often still failed
- 2001: a small group of developers wrote the **Agile Manifesto** instead
- Idea: work in small steps, involve the customer often, and be ready to change the plan

A reaction against heavy process — not against being disciplined.

## 1990s–2000s: software moves onto the internet

- **Open source** — software like Linux, built and improved by volunteers all over the world, free for anyone to use
- The internet turns software into a service that is always running, not a product you install once
- Teams start testing and combining their code automatically, many times a day, instead of once every few months

Software now has to work for huge numbers of users, all the time.



**There is no cloud**  
it's just someone else's computer

- **Cloud computing** — rent computers over the internet instead of buying and maintaining your own
- **DevOps** — the people who build the software also help run it, instead of handing it off to a separate team
- **Microservices** — split one big system into many small, independent pieces that can be changed on their own

Releasing new software becomes routine, not a rare, risky event.

## 2020s: AI-assisted development



- Software development disproportionately affected by recent advances in AI
- Tools like GitHub Copilot and Claude Code can now write, explain and fix code for you
- The bottleneck is now reviewing, understanding, verifying and maintaining large volumes of generated code
- Do they help with the hard part — deciding *what* to build?

# The software crisis

---

# The software crisis

The problem(s) acknowledged at the same NATO conference in 1968:

- Projects were routinely late and/or over budget
- Software was unreliable, inefficient or otherwise of low-quality
- It did not do what users needed
- It was too expensive to maintain and change
- Large projects were being abandoned and cancelled

## Dijkstra's diagnosis



*The major cause of the software crisis is that the machines have become several orders of magnitude more powerful! To put it quite bluntly: as long as there were no machines, programming was no problem at all; when we had a few weak computers, programming became a mild problem, and now we have gigantic computers, programming has become an equally gigantic problem.*

— Edsger Dijkstra, The Humble Programmer (EWD340), Communications of the ACM

## Moore's law

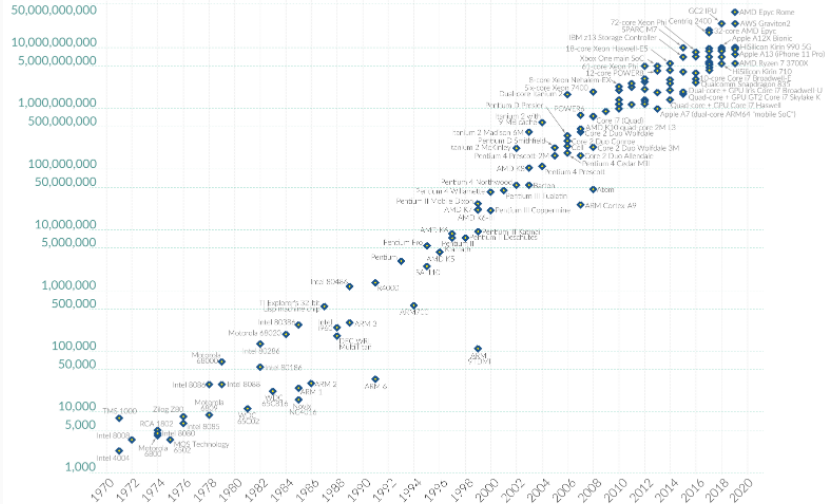
Moore's Law: The number of transistors on microchips doubles every two years

Moore's law describes the empirical regularity that the number of transistors on integrated circuits doubles approximately every two years.

This advancement is important for other aspects of technological progress in computing – such as processing speed or the price of computers.

Our World  
in Data

## Transistor count





# Is the software crisis over?

- We are dramatically better at building software than in 1968
- Software development practices improved enormously
- Programming languages, tools and hardware improved
- Software systems became vastly more capable
- Yet large software projects still fail surprisingly often

[https://en.wikipedia.org/wiki/List\\_of\\_failed\\_and\\_overbudget\\_custom\\_software\\_projects](https://en.wikipedia.org/wiki/List_of_failed_and_overbudget_custom_software_projects)

*The problem changed — but did it go away?*

# Software failures can be spectacular

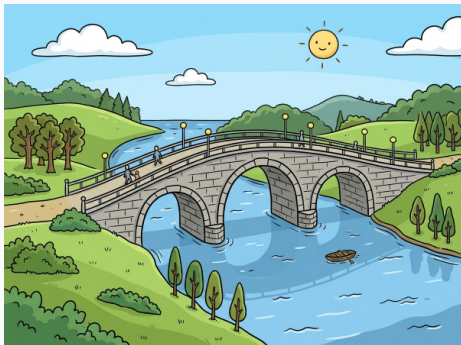
- **Ariane 5 Flight 501 (1996)**
  - Launcher exploded ~40 seconds after launch
  - Cause included multiple software specification/design errors
  - ~€370 million of hardware was lost
- **FBI Virtual Case File (2001–2005)**
  - Major attempt to modernize FBI case management
  - Project was eventually cancelled
  - About \$170 million spent
- **Healthcare.gov (2013)**
  - Budget of \$292 million
  - Severe problems at launch
  - Millions of users unable to register



# Why building software is different from traditional engineering

---

# Why can't we just engineer software like a bridge?



Building a bridge



Building software

Both involve engineering, design, planning and testing.

**But the nature of the problems is fundamentally different.**

# Two kinds of difficulty

## Essential difficulties

Difficulties that are inherent in the problem itself.

- Complexity
- Conformity
- Changeability
- Invisibility

## Accidental difficulties

Difficulties introduced by the way we build software.

- Primitive tools
- Awkward languages
- Manual processes
- Slow compilation
- Poor development environments
- ...

*Brooks: progress on accidental difficulties is possible.  
There is no “silver bullet” for essential difficulties.*



*There is no single development, in either technology or management technique, which by itself promises even one order of magnitude improvement within a decade in productivity, in reliability, in simplicity.*

*— No Silver Bullet — Essence and Accident in Software Engineering, 1986*

### Software systems are extremely complex objects.

- A large system may contain millions of interacting elements
- Components can interact in almost arbitrary ways
- Small changes can have unexpected consequences
- Complexity grows faster than the number of components
- There is no natural “simplification” imposed by physics

*The difficult part is often not writing the individual components,  
but understanding how they interact.*

**Software must conform to an enormous number of external constraints.**

Operating systems

Regulations

Existing systems

Human expectations

APIs

Business rules

Data formats

User interfaces

Hardware

Organizational  
processes

Protocols

Legacy behaviour

*Much of this complexity exists outside the software itself.*



Software is expected to change continuously.

Users

“Can we also...?”

Business

New markets  
New priorities

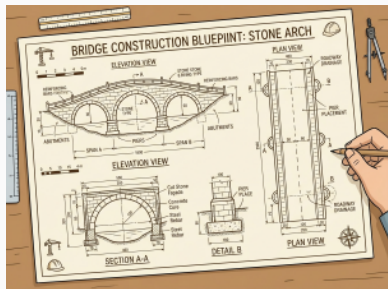
Environment

New hardware  
New platforms  
New regulations

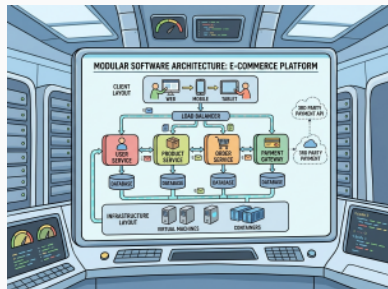
A software system is rarely “finished”.

*We don't just build software.  
We continuously evolve it.*

## Essential difficulty #4: Invisibility



A bridge has a physical structure



Software's structure is abstract

- Software architecture cannot be directly observed
- Dependencies are often invisible
- A system can look “fine” while being internally chaotic
- We need representations: diagrams, models, code, tests, metrics

## Optimizing for learning and managing complexity

---

# Software development is a learning problem

At the beginning of a software project, we don't know everything:

- What do users actually need?
- Will our design work?
- Which architecture is appropriate?
- Which technology will work best?
- Where are the difficult edge cases?
- Which assumptions are wrong?

**We learn by building software.**

Therefore:

Design the development process to maximize  
**useful feedback and learning.**

# Optimizing for learning

Farley identifies five key techniques:

## 1. Iteration

- Work in small cycles

## 2. Feedback

- Find out quickly whether we're right

## 3. Incrementalism

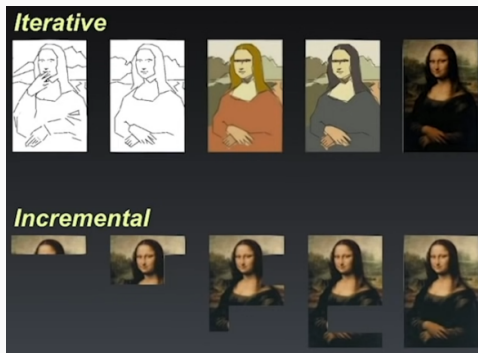
- Build the system piece by piece

## 4. Empiricism

- Base decisions on evidence

## 5. Experimentation

- Treat uncertain ideas as hypotheses to test



# Managing complexity

Farley's five key techniques:

## 1. **Modularity**

- Split the system into understandable parts

## 2. **Cohesion**

- Keep related responsibilities together

## 3. **Separation of concerns**

- Keep different kinds of problems separate

## 4. **Abstraction & information hiding**

- Hide unnecessary implementation details

## 5. **Loose coupling**

- Minimize dependencies between parts

## Optimize for learning

Development process

Requirements

Prototyping

Testing

Code reviews

CI/CD

Frequent feedback

## Manage complexity

Architecture

Modularity

Abstraction

Design principles and patterns

Documentation

Refactoring

We will cover most of these topics in this course.

## Summary

---



# Why is it hard?

- There is no single "right way" to engineer software systems
  - The combination of constraints within a project is probably unique
  - There is no universal "best practice"
- Scale
  - Both systems and organizations producing them are larger
  - Systems rarely exist in isolation
- Communication
  - Distributed
  - Lack of well-understood representations of software
- The rate of change in the business is higher

# Key aspects of software engineering

- Requirements reflecting the real customer needs
- Suitable design
- Effective and predictable construction
- Reliable quality assurance
- Safe deployment
- Cost-effective maintenance
- Organized and predictable project execution

In practice SE is mostly about processes, methodologies, and tooling.

## Key takeaways

- Software engineering is about building teams and evolving high-quality software, while working under constraints unique to their circumstance.
- Writing code  $\neq$  building software
- There is no single "right way" to engineer software systems
- But we can identify principles and practices that help us manage complexity and learn faster

Questions?